

Warum es schön ist, zitiert zu werden: Quasiquoting für Prolog

Jan Wielemaker `{ } ^ { 1 } $` und Michael Hendricks `{ } ^ { 2 } $ $ { }`
`{ } ^ { 1 } $` Web- und Mediengruppe, VU Universität Amsterdam, De
Boelelaan 1081a, 1081 HV Amsterdam, Niederlande,
J.Wielemaker@vu.nl

`{ } ^ { 2 } $` Hendricks Solutions, LLC, Hamna, Wyoming, USA,
michaelendrix.org

arXiv:1
308.3
941v1
[cs.PL]
19.
Aug.
2013

Die Unterstützung von dynamischer Programmierung, Metaprogrammierung und Textverarbeitung unter Verwendung kontextfreier Grammatiken macht die Sprache sehr geeignet für die Definition von domänenspezifischen Größen (DSL) sowie für die Analyse, das Refactoring oder die Generierung von Ausdruckszuständen in anderen (Programmier-)Sprachen. Bekannte DSLs sind die DCG-Notation (Definite Clause Grammar) und Constraint-Sprachen wie CHR. Diese Erweiterungen verwenden Prolog-Operatordeklarationen und die `{ } ^ { 2 } $`-Notation, um eine gute Syntax zu realisieren. Wenn externe Sprachen wie HTML, SQL oder JavaScript ins Spiel kommen, genügt es für Operatoren nicht mehr, Ausschnitte dieser Sprachen in eine Prolog-Quelldatei einzubetten. Darüber hinaus bietet Prolog eine schlechte Unterstützung für das Zitieren langer Textfragmente. Hszkell führte Quasi-Zitate ein, um dieses Problem zu lösen. In diesem Artikel haben wir den Haskell-Mechanismus für die Quasi-Zitaterstellung nach Prolog "portiert". Wir zeigen, dass dies sauber gemacht werden kann und dass quasi das Zitieren die oben genannten Probleme lösen kann.

1 Einleitung

Prolog wird häufig für Aufgaben verwendet, bei denen Codeausschnitte verwaltet werden müssen, die in der Syntax einer externen Sprache wie HTML, SQL oder JavaScript geschrieben wurden. Ein solcher Code Snippets entsprechen oft nicht der Standard-Prolog-Syntax [5]:

- Die Verwendung von Operator-Syntax, die nur Präfix-, Infix- und Postfix-Operatoren enthält, ist unzureichend. Betrachten Sie zum Beispiel die JavaScript-case-Anweisung.- Grundlegende lexikalische Primitive sind inkompatibel. Betrachten Sie beispielsweise Textinhalte in HTML-Dokumenten.
- Der Prolog-Begriff, der sich aus der Analyse eines Ausdrucks ergibt, reicht nicht aus, um die Absicht in der Zielsprache wiederherzustellen. Betrachten Sie z. B. Bezeichnernamen, die mit einem Großbuchstaben beginnen, der als Prolog-Variable gelesen wird.

In [5] behaupten wir, dass einige einfache Sprachen gut abgedeckt werden können, indem die Prolog-Syntax mit Hilfe von Operatoren optimiert wird, während es möglich ist, akzeptable Darstellungen für einige andere, z.B. XML-basierte Sprachen, zu realisieren, indem eine Prolog-Syntax verwendet wird, um das (einfache) Datenmodell dieser Sprache darzustellen. In anderen Fällen kann die Einbettung externer Sprachen teilweise durch semantische Transformationen unterstützt werden. Aufrufe von Prolog-Prädikaten, deren Funktor mit einer Tabelle in einem RDBMS übereinstimmt und deren Arität mit dem

Anzahl der Spalten kann in SQL-Abfragen übersetzt werden [2]. In [5] schlagen wir eine Reihe von syntaktischen Erweiterungen vor, um den Bereich der Sprachen zu erweitern, die mit einem der oben genannten Mittel unterstützt werden können. In diesem Artikel schlagen wir eine Lösung für komplexe externe Sprachen vor, für die keine der oben genannten eine zufriedenstellende Einbettung erreicht. Die Lösung wird Quasi-Zitate genannt, wobei wir die Integration in die Sprache von Haskell [4] übernehmen. Ein Haskell-Quasi-Zitat hat als syntaktisches Element eine Syntaxkennung (ein Funktionssymbol) und einen Ausschnitt aus zitiertem Text. Die Funktion, die mit der Syntaxkennung verbunden ist, wird während des Parsens aufgerufen und kann den Text manipulieren und so bearbeiten, dass er für die weitere Verarbeitung durch Haskell geeignet ist. Es ist eine gute Praxis für die Funktion, einen abstrakten Syntaxbaum zu erstellen, indem sie den Text gemäß den Regeln der externen Sprache parst. Wie wir in diesem Artikel sehen werden, können auch einfachere Ansätze ausreichend sein. Dieser Artikel ist wie folgt aufgebaut. Nach der Untersuchung verwandter Arbeiten werden wir die Problemstellung erneut aufgreifen und beschreiben, warum die aktuelle Unterstützung in Prolog nicht ausreicht, um dieses Problem zufriedenstellend zu lösen. In Abschnitt 6 stellen wir den Begriff des Quasi-Zitierens vor und wie er sich in die Verwaltung von Zeichenketten einfügt, die Datenstrukturen aus anderen Sprachen darstellen. In Abschnitt 7 stellen wir die konkrete Lösung vor, die für SWI-Prolog vorgeschlagen wird. Es folgen Beispiele, zukünftige Arbeiten und die Schlussfolgerungen.

2 Verwandte Arbeiten

Quasi-Zitate werden zum Einbetten von Code-Schnipseln in vielen 'Skript'-Sprachen verwendet, da sie einen leicht verständlichen Mechanismus darstellen, um lange Zeichenketten oder ASTs (Abstract Syntax Trees) mit eingebetteten Variablen zu erstellen, denen Werte zugewiesen werden, die von der Umgebung bestimmt werden. Der reine Textersatz birgt das Risiko von Injection-Angriffen, was die Forschung in Richtung sichererer Template-Engines ausgelöst hat. `{ }^3$` Insbesondere moderne Skriptsprachen wie Python und Ruby bieten eine reichhaltige Palette solcher Techniken. Dieser Artikel folgt der Argumentation und Technik, die in "Why It's Nice to be Quoted: Quasiquoting for Haskell" [4] beschrieben wird. Wir verweisen auf diesen Artikel für weitere Hintergrundinformationen. Ein weiteres interessantes System ist Camlp4 [1], das eine Parsing- und 'Anti-

Parsing'-Infrastruktur für OCaml bereitstellt. Beide Ansätze ermöglichen die Erweiterung der Hostsprache. Der Hauptunterschied zu unserem Ansatz besteht darin, dass wir lediglich einen Hook in den Prolog-Parser vorschlagen, der eine intelligente Verarbeitung von Quasi-Zitaten ermöglicht, ohne vorzuschreiben, wie diese Zitate verarbeitet werden.

3 Die Notwendigkeit, Code-Schnipsel externer Sprachen in Prolog einzubetten

Wir werden diese Notwendigkeit anhand eines Beispiels aus den Webseitengenerierungsfunktionen von SWI-Prolog [6] erläutern. Webseiten sind keineswegs der einzige Ort, an dem diese Probleme auftreten [4], aber die Domäne ist vielen Programmierern vertraut und von besonderem Interesse für uns

```
ootnotetext{$ }^3$ TAL (http://www.owlfish.com/software/simpleTAL/tal-guide.html)
Cheet2h (http://www.cheetahtemplate.org/examples.html) Django
(http://docs.djangoproject.com/en/dev/ref/templates/) Jinja2 (http://jinja-pocoo.org/docs/templates/).}
```

denn das Generieren von Webseiten ist ein wichtiges Anwendungsgebiet für SWI-Prolog. Eine moderne Webseite besteht in der Regel aus HTML, das die anfängliche DOM-Struktur erstellt, CSS, das das Styling bereitstellt, und JavaScript zum Erstellen interaktiver Komponenten. Unsere Lösung zur Generierung von HTML ähnelt PiLLoW [3] und basiert auf der Darstellung der HTML-DOM-Struktur als verschachtelter Prolog-Begriff. Die SWI-Prolog HTML-Bibliothek zur Seitengenerierung bietet gute Lösungen für

- Generieren der Seitenstruktur.- Definition und Wiederverwendung von DCGs, die Seitenfragmente generieren.
- Erstellen Sie Verweise (URLs) auf andere Seiten auf dem Server.
- Modulares Einbinden von Ressourcen, wie z.B. Style- und JavaScript-Dateien inkl. Abhängigkeitsverfolgung und automatischer Sortierung durch Berechnung einer Teilreihenfolge auf Basis der abhängigkeiten.

Seiten, die mit dieser Infrastruktur generiert werden, sind garantiert syntaktisch korrekt, unabhängig von den Daten, aus denen die Seite erstellt wird. Dieser integrierte Schutz vor Injection-Angriffen gilt als wichtiger Vorteil der Serverinfrastruktur und als eine Eigenschaft, die wir bei der Integration von JavaScript in die Seite beibehalten möchten.

In früheren Veröffentlichungen [5, 6] haben wir bereits zwei fehlende Teile identifiziert: (1) längere (HTML) Textfragmente und (2) JavaScript-Fragmente. Das erste Problem kann mit PWP [4] gelöst werden. PWP wurde von Richard O'Keefe entwickelt und kann als der Gegner angesehen werden

Seite des oben beschriebenen Prolog-basierten Seitengenerators. Die Seite wird in XML beschrieben und reservierte XML-Elemente und -Tags lösen die Generierung dynamischer Seitenkomponenten basierend auf der Auswertung von Prolog-Zielen aus. Wie der Prolog-Seitengenerator schützt PWP vor Injektionsangriffen, da es Lösungen aus Prolog als Daten betrachtet, die zur Erweiterung des XMI-DOM verwendet werden, das anschließend serialisiert wird, um die endgültige Seite zu generieren. PWP vereinfacht die Spezifikation von Seiten mit großen Textmengen, aber das Regelformat ist weniger natürlich und es fehlt die Modularität des Prolog-basierten Generators.

Wir haben mit JavaScript gekämpft und verschiedene Ansätze ausprobiert. Derzeit gibt es eine große Anzahl von JavaScript-Frameworks, die "Widgets" für die Verwendung auf Webseiten bereitstellen. Beliebte Beispiele sind YUI^5 und jQuery^6 . Das Einbinden dieser Widgets erfordert in der Regel die Erstellung einer DOM-Struktur, die ein 1d-Attribut trägt und die Daten darstellt (z. B. kann ein Menü als ul-Liste oder ein erweiterter Texteditor als Textbereich dargestellt werden). Dieses DOM-Element wird in das Ziel-Widget transformiert, indem eine JavaScript-Initialisierungsfunktion aufgerufen wird, die das DOM des Zielelements ändert und Ereignishandler installiert, die das dynamische Verhalten des Elements bereitstellen. Die Initialisierungsfunktion verwendet häufig ein Konfigurationsobjekt (ein JavaScript-Objektliteral, $\{\text{namevalue}, \dots\}$), wobei einige der Werte Callback-Funktionen sind, die als JavaScript-Lambda-Funktionen dargestellt werden. Das HTML-Framework ist gut gerüstet, um das anfängliche DOM zu erstellen und die JavaScript- und CSS-Ressourcen mithilfe seines Abhängigkeitsframeworks abzurufen. Der Initialisierungsaufwurf

Prolog Wohlgeformte Seiten, [http://www.swi-prolog.org/pldoc/package/\\$\mathrm{sgml} . \mathrm{html}](http://www.swi-prolog.org/pldoc/package/$\mathrm{sgml} . \mathrm{html}) $\mathrm{YUILibrary.com}$ $\mathrm{jQuery.com}$

ist einfach genug zu abstrahieren, es sei denn, das Konfigurationsobjekt enthält Lambda-Funktionen. Diese sind häufig erforderlich, um JSON-Daten vom Server in die gewünschte Visualisierung zu massieren und AJAX-basierte Vorgänge zu unterstützen, wie z. B. das Berechnen von Autovervollständigungskandidaten. Es ist durchaus üblich, dass solche Objekte Dutzende von Zeilen JavaScript benötigen. Die Darstellung von JavaScript-Lambda-Funktionen erfordert Unterstützung für die gesamte Komplexität der Sprache. Eine Möglichkeit, dies zu tun, besteht darin, das JavaScript-Fragment als eine Liste von Literalen darzustellen, die mit DCG-Regeln gemischt sind, die von Prolog generierte Inhalte einfügen, z. B. URLs oder JavaScript-Literalobjekte. Dies führt zu Code wie unten, den wir als schwer auf einer Tastatur zu tippen, schwer lesbar und, weil er so schwer zu lesen ist, oft anfällig für Syntaxfehler halten. Schließlich ist der folgende Code anfällig für Injektionsangriffe, es sei denn, wir übergeben die Variablen URL und Id an eine Grammatik, die gültigen JavaScript-Zeichenfolgeninhalt aus einem beliebigen Prolog-Atom generiert.

```
"['s.ajax(f url: "",URL,"",'\n',
  ' data: { id: "",Id,"" $\\mathrm{Sn}^{\\prime}$,
  '$1 \\mathrm{n}^{\\prime}$, 'b$) ;
  \\mathrm{n}^{\\prime}$]"
```

Wir haben mehrere Designs ausprobiert, um die oben genannten Punkte zu verbessern, von denen wir keines für zufriedenstellend hielten. Im Folgenden finden Sie die wichtigsten Richtungen, die wir ausprobiert haben.

- Abstrahieren Sie weg. Dies impliziert die Verwendung der Funktionen zur Generierung von HTML-Seiten, um z. B. einen JavaScript-Aufruf zu generieren, eine Variable aus Prolog-Daten zu initialisieren usw. Um überschaubar zu bleiben, ist ein relativ hohes Maß an Abstraktion erforderlich, das sich an dem verwendeten JavaScript-Framework orientiert. Dies ist problematisch, da es für den Programmierer schwierig ist, die JavaScript-Beispiele aus der Framework-Dokumentation mit dem Prolog-Code in Beziehung zu setzen. Obwohl es möglich ist, einige der Lambda-Funktionen zu abstrahieren, gibt es zu viele Variationen, um sie alle zu behandeln.

- Legen Sie das JavaScript in eine separate Datei. Dadurch entstehen viele kurze Dateien, die in der Regel nur eine bestimmte generierte HTML-Seite unterstützen, da die Details des erforderlichen JavaScripts, wie z. B. die zu adressierenden Speicherorte auf dem Server, von Seite zu Seite variieren. Da es keine formale Beziehung zwischen den beiden Seiten gibt, ist es schwierig, sie in Beziehung zu setzen und synchron zu halten.- Erstellen Sie anwendungsspezifische JavaScript-Ressourcen, die konfiguriert werden können, ohne dass Lambda-Funktionen im Konfigurationsobjekt erforderlich sind. Das Problem dabei ist, dass die Anwendungsentwickler ihre eigene Verfeinerung der externen Widget-Bibliothek erstellen, die Verständnis und Dokumentation erfordert, während die extern bereitgestellten Bibliotheken bereits ein recht hohes Niveau haben. Mit anderen Worten, die neu erstellte Schicht fügt hauptsächlich neue Komplikationen für die Verwaltung und das Verständnis des Codes hinzu.

Wir sind zu dem Schluss gekommen, dass (1) wir einen Mechanismus benötigen, der es ermöglicht, JavaScript in den Quellcode der Prolog-Seitengenerierung einzubinden, (2) die Darstellung von JavaScript in der Prolog-Quelle einfach auf einer Tastatur einzugeben und zu verstehen sein muss, (3) das generierte JavaScript gegen Injektionsangriffe ohne explizite Aufrufe zur Codierung von Daten geschützt sein sollte und (4) die syntaktische Validierung der generierten Fragmente wahrscheinlich die Produktivität verbessert.

4 Verwendung der Prolog-SyntaxZunächst untersuchten wir, inwiefern die in [5] vorgeschlagenen Syntaxerweiterungen dazu beitragen würden, JavaScript auf natürliche Weise mit der Prolog-Syntax darzustellen. Wir haben die Unterstützung für die leeren Argumentlisten (z.B. Name (1), Array-Notation und Funktionskörper mit $\{\{\text{Idots}\}\}$) realisiert. Diese wurden nach einem Vorschlag von Jose Morales gelöst, der den Begriff der Operatoren auf Listen und Blöcke in geschweiften Klammern erweitert. Zusätzlich zu unserer Hoffnung, die JavaScript-Unterstützung zu verbessern, werden Listen mit Listenabonnements und Attributlisten in geschweiften Klammern mit B-Prolog und ECLiPSe verwendet. Wir haben einen Prototyp realisiert, der einen Großteil der JavaScript-Syntax ausdrücken kann. Folgende Probleme sind jedoch noch gelöst:

- Es erfordert ! und. als Operatoren definiert werden. Es ist bekannt, dass diese Operatoren Mehrdeutigkeitsprobleme verursachen.- Möglicherweise ist eine Anführungszeichen von Bezeichnern erforderlich (z. B. 'String' (. .))- Ein spezielles Symbol wird benötigt, um Bezeichner von Zeichenketten zu unterscheiden. ("string" kann nicht verwendet werden, da es sich um eine Liste von ganzen Zahlen handelt).
- Es sind Korrekturen in Bezug auf die Räume erforderlich. Nach Funktionssymbolen und Schlüsselwörtern wie while ist kein Leerzeichen zulässig. An einigen Stellen sind zusätzliche Leerzeichen erforderlich, um zu verhindern, dass Prolog zwei JavaScript-Token als ein einzelnes Prolog-Token liest.- Einige Symbole kollidieren mit Prolog. Betrachten | oder %, die in der Prolog-Syntax eine völlig andere Rolle spielen.- Einige JavaScript-Snippets können in gültige Prolog-Syntax umgewandelt werden, aber das resultierende AST ist mehrdeutig. Zum Beispiel unterscheidet sich ++a von $\mathrm{a}++$, aber beide ergeben den Prolog-Term ++ (a). Wir glauben, dass das Ergebnis brauchbarer gewesen wäre als die Verwendung von Prolog-Atomen in Anführungszeichen für kurze (1-10 Zeilen) JavaScript-Snippets, die einfache Lambda-Funktionen wie das Erstellen trivialer AJAX-Callbacks enthalten. Unser größter Zweifel ist, dass es schwierig ist,

Vermitteln Sie die Einschränkungen an gelegentliche Prolog-Benutzer, was es wahrscheinlich macht, von einer Ausgabe überrascht zu werden, die von den Erwartungen abweicht und nur durch eine tiefe Kenntnis der Prolog-Syntax und der zugrunde liegenden Begriffsdarstellung verstanden werden kann.

5 Die drei Probleme

Wenn wir JavaScript nicht in der Prolog-Syntax mit benutzerdefinierten Operatoren einbinden können, besteht die andere Möglichkeit darin, Text in Anführungszeichen zu verwenden. Dazu müssen wir drei orthogonale Probleme lösen:

1. Erlauben Sie lange zitierte Texte. Der aktuelle (ISO) Prolog bietet nur Text in einfachen und doppelten Anführungszeichen, die denselben Regeln folgen. Besonders
 - Text in Anführungszeichen darf nicht mehr als eine Zeile umfassen, es sei denn, der Zeilenumbruch wird mit einem umgekehrten Schrägstrich versehen.
 - Der umgekehrte Schrägstrich und das Anführungszeichen müssen mit einem umgekehrten Schrägstrich maskiert werden.- Wenn ein gutes Layout der Ausgabe wünschenswert ist, gibt es keine Möglichkeit, Fortsetzungszeilen in der Prologquelle so einzurücken, dass der Einzug nicht in der Ausgabe angezeigt wird.

Mit diesen Einschränkungen muss auf das einfache Kopieren und Einfügen von Beispielcode ein langwieriger Prozess folgen, bei dem der Text an die Prolog-Syntax angepasst wird, während das Gegenteil der Fall ist zum Testen des Codes benötigt, z. B. in einer JavaScript-Konsole.² Verknüpfen Sie Prolog-Variablen (Daten) mit Referenzen im zitierten Material.³ Richten Sie eine `sqfe`-Methode ein, um Daten in die Vorlagenzeichenfolge einzubetten.

Diese Probleme sind unabhängig voneinander. Wir verdeutlichen dies mit einem kleinen HTML-Beispiel, das eine Seite mit der aktuellen Uhrzeit darstellt. Wir verwenden ein HTML-Skelett, um den Vergleich fair zu gestalten. In reinem Prolog könnte dies mit dem folgenden Code-Snippet erreicht werden:

```
...
get_t1me (Jetzt),format_time(atom(Datum),'%+',
Jetzt),print_html([Datum-Datum], '<h1>Meine
Digitaluhr</h1>\n\
<p>Es ist jetzt {{date}}</p>')."
```

Die Aufgabe von `print_html` / `2` besteht darin, die Vorlagenvariable `date` mit der Prolog-Variablen `Date` (drittes Element oben) in Beziehung zu setzen, den lexikalischen Kontext von `{{date}}` zu bestimmen, um die richtige Codierung zu kennen, die durchgeführt werden muss, diese Codierung auf dem Prolog-Atom zu realisieren und die richtige Ausgabezeichenfolge zu erstellen. Dies ist der zweite Punkt in der obigen Liste. Im Idealfall möchten wir einen Fehler, wenn das HTML-Fragment von Anfang an falsch formatiert ist. In reinem ISO-Prolog müssen alle Arbeiten zur Laufzeit ausgeführt werden. Viele Prolog-Systeme bieten eine Zielerweiterung/² oder ähnliches, die es ermöglicht, die Analyse des Templates bei Kompilierzeit. Was auf unserer Wunschliste bleibt, ist, (1) `[date-Date]` loszuwerden und (2) komplizierte Escape-Sequenzen für Sprachen loszuwerden, die häufig Anführungszeichen oder Backslashes verwenden. Wir schlagen vor, beide Probleme mit Hilfe von Quasi-Zitaten für Prolog zu lösen.

6 Quasi-Zitate Quasi-Zitate finden ihren Ursprung in der Linguistik `{{}^{7}}` und führen Variablen in textuelle Ausdrücke ein. Sie werden häufig in Skriptsprachen verwendet. Zum Beispiel:

```
"eine-Welt; echo "Hallo $a" // Un1x shell$a - "Welt"; print "Hallo $a"; Perl"
```

Dieser Ansatz ist für den (unerfahrenen) Programmierer natürlich. Leider funktioniert es für Prolog nicht, da nichts im normalen Prolog-Kompilierungsprozess Zugriff auf die Namen von Variablen bietet. Darüber hinaus ist das Einfügen von Klartext eine direkte Einladung zu Injektionsangriffen. Wie in Abschnitt 5 beschrieben, muss der Code zum Ersetzen von Vorlagen die Syntax kennen, um eine sichere, d.h. ordnungsgemäß maskierte Einfügung der Variablen durchzuführen.

Haskell-Quasi-Anführungszeichen lösen das Injektionsproblem (das in [4] als Typisierungsproblem beschrieben wird), indem sie einen Syntaxbezeichner direkt mit den zitierten Daten verknüpfen. Der Syntaxbezeichner ist mit einer Funktion verknüpft, die das zitierte Material in der Regel in einen abstrakten Syntaxbaum analysiert, der die Zielsprache darstellen kann.

`{{}^{7}}` <http://en.wikipedia.org/wiki/Quasi-quotation>

7 Quasi-Zitate in Prolog

Das Realisieren von Quasi-Zitaten erfordert ein syntaktisches Konstrukt, das (1) lange Zeichenketten in Anführungszeichen liefert, (2) das Zitat mit einem Prädikat verknüpft, das entsprechend den Anforderungen der Verarbeitung der externen Sprache darauf reagieren kann, und (3) Zugriff auf das Variablenwörterbuch der Klausel bietet. Es gibt nur wenige Optionen, um Prolog eine Syntaxerweiterung hinzuzufügen, da Prolog-Symbolzeichen zu einem Atom zusammenkleben, dem Operatoreigenschaften zugewiesen werden können. Laut Ulrich Neumerkel ist `{ } ^ { 8 }` das Aufnehmen von Sequenzen von Prolog-Solocharakteren, die keine Namen bilden, ein guter Ausgangspunkt. Dies ermöglicht unter anderem `{ | \ldots | }`. Kombiniert mit einem Begriff, der die Syntax identifiziert, schlagen wir die folgende Syntax für Prolog-Quasi-Zitate vor, wobei Syntax ein beliebiger (aufrufbarer) Prolog-Begriff ist:

`{ [Syntax-identifizier | quoted-material ] }` In dem eher unwahrscheinlichen Fall, dass `{ \mid }` in das zitierte Material eingebettet werden muss, kann dies auf zwei Arten realisiert werden: (1) Definieren Sie das Prädikat, das das Zitat verarbeitet, um eine Escape-Sequenz zu respektieren, oder (2) nutzen Sie die vorhandene Flexibilität der Zielsprache, um `{ \mid }` zu vermeiden. Ein Beispiel für (1) könnte die Einführung des Mappings `{ \backslash \backslash \rightarrow { $, $ \backslash } \rightarrow { $, $ \backslash } }` sein, wonach der Benutzer `{ \mid \backslash }` schreiben kann. Ein Beispiel für (2) kann sein, ein Leerzeichen zwischen den beiden einzufügen, wenn dies die Semantik nicht ändert, oder `{ \backslash u 007 c $ }` in eine Zeichenkette zu schreiben, wenn die Zielsprache `\uxxxx escapes` innerhalb von Zeichenketten unterstützt. Orthogonal zur Syntax ist die Abbildung des zitierten Materials auf einen Prolog-Begriff,

Dabei kann es sich um eine vollständige AST des Snippets oder eine vereinfachte Darstellung handeln, wie in Abschnitt 8.2 verwendet. Diese Zuordnung wird durch den Syntax-Identifizier definiert, einen aufrufbaren Begriff, den wir von nun an als Syntax bezeichnen. Gleichzeitig führt Quasi Quotation Variablen aus der Umgebung mit dem zitierten Material zusammen (Prolog). Quasi-Angebote werden wie folgt verarbeitet:

- Das Prädikat `read/ $ \mathbf{f} { 1 } $` wird geändert, um den Syntaxbegriff und den zitierten Text zu erkennen.- Nachdem die normale Verarbeitung abgeschlossen ist, führt `read/1` den folgenden Aufruf aus: `"call(+SyntaxName, +Content, +SyntaxArgs, +VarDict, -Result)"` Hier ist `SyntaxName` der Funktor des Syntaxbegriffs und `SyntaxArgs` ist die Liste der Argumente, d.h. `Syntax ... [SyntaxName|SyntaxArgs]`. In Abschnitt 8 werden wir sehen, warum es nützlich ist, den Funktor von den Argumenten zu trennen. Der Inhalt ist ein undurchsichtiger Griff für das zitierte Material, wie wir später sehen werden. `VarDict` ist `Name = Var`-Liste, die dem Lesen von Begriff/3 und Schreiben von Begriff/3 entspricht. Das Ergebnis wird durch den Aufruf des deterministischen Prädikats `SyntaxName/4` bestimmt. Das Prädikat `read/1` fügt `Result` an der Stelle des Quasi-Zitats im Ausgabeterm ein.- Der `Result`-Term (und damit das Quasi-Zitat) muss ein Ziel sein (siehe Abschnitt 8.3) oder als Argument eines Ziels erscheinen, das das Ergebnis verarbeitet (oft serialisiert) (oft ein AST). Zum Beispiel ist die Antwort `html page/3` in Abschnitt 8.1 darauf ausgelegt, das HTML DOM (AST) zu serialisieren, das von dem HTML-Parser erzeugt wird, der vom Quasi-Anführungszeichen aufgerufen wird. `{ } ^ { \text { 5 } } $ "https://lists.iai.uni-bonn.de/pipermail/swi-prolog/2013/$010422 . \mathrm{html} $ $`

Wir stellen zwei Unterstützungsprädikate für `SyntaxName/4` bereit, um das Ergebnis zu verarbeiten. Beachten Sie, dass es keine Einschränkungen gibt, wie das aufgerufene Prädikat den zitierten Text mit den Syntaxargumenten und dem Variablenwörterbuch kombiniert, um den endgültigen Begriff zu konstruieren. `phrase from quasi quotation(: Grammatik, + Inhalt)` Ruft die Grammatik `Grammatik` in der Liste auf, die aus `Inhalt` gebildet wird. Dieses Prädikat verwendet die in [7] beschriebene `purs`-Eingabebibliothek, um den Inhalt zu analysieren. Syntaxfehler können mit dem nicht-terminalen `syntax_error//1` ausgelöst werden, der eine genaue Syntaxposition erzeugt, die aus der Datei, der Zeilennummer, der Zeilenposition und der Zeichenanzahl besteht. `with_quasi_quotation Input(+Inhalt, -Stream, Ziel)`

Ruft `Goal` für den Prolog-Stream `stream` auf. Die Informationen zur Stream-Position spiegeln den Speicherort in der Quelldatei wider, mit Ausnahme der Byte-Anzahl.

8 Beispiele für die Verwendung von Quasi-Anführungszeichen in Prolog

In diesem Abschnitt stellen wir drei Beispiele zur Veranschaulichung von Quasi-Zitaten zur Verfügung. Das erste Beispiel (HTML) basiert auf den bestehenden SWI-Prolog-Bibliotheken zum Parsen und Serialisieren von HTML. Das zweite Beispiel zeigt, wie JavaScript sicher (aber mit eingeschränkter Unterstützung für die Syntaxprüfung) gehandhabt werden kann, indem nur das Zitat tokenisiert wird. Das dritte Beispiel betrifft SPARQL, `{ }^9$` demonstriert den Wert des Quasi-Zitat-Ansatzes in einem Szenario, in dem das geparsete Quasi-Zitat als Prolog-Abfrage verwendet wird.

8.1 Quasi HTML zitieren Das erste, vollständige Beispiel veranschaulicht die sichere Einbettung von langen HTML-Texten in eine Prolog-Webseite. Der Code des Quasi-Anführungszeichens ist in Abbildung 2 dargestellt. Zunächst geben wir einen voll funktionsfähigen Webserver an, der auf diesem Quasi-Zitat in Abbildung 1 basiert. Wir stellen fest, dass das zitierte Material mehrere Zeilen enthalten kann, keine Zeilenenden erfordert und Anführungszeichen enthalten kann. Der Inhalt des zitierten Materials ist gültiges HTML, und da ein Redakteur die Indikatoren `{ }^mid \mathrm{html}$` und `{ }^mid}$` leicht erkennen kann, ist es für Entwicklungstools nicht schwierig, Unterstützung für den eingebetteten HTML-Code bereitzustellen, wie z. B. Hervorhebung, Einrückung oder Vervollständigung. Das HTML-Anführungszeichen ist so definiert, dass Prolog-Variablen, die ersetzt werden, auf diejenigen beschränkt werden, die als Argumente für den HTML-Syntaxindikator angezeigt werden (Datum in Abbildung 1). Die Konvention zur Übergabe von Variablen, die explizit ersetzt werden müssen, hat folgende Vorteile: (1) sie vermeidet eine Warnung der Singleton-Variablen am Datum `{ }^10$`, (2) Prolog-Klauseln haben eine relativ große Anzahl von Variablen aufgrund des Fehlens einer funktionalen Notation, die Variablen vermeiden kann, und destruktive Zuweisungen, die die Wiederverwendung einer Variablen ermöglichen, (3) sie macht

die Ersetzung expliziter und (4) es dem Zitatgeber ermöglicht zu überprüfen, ob alle beabsichtigten Beziehungen hergestellt wurden. Das HTML-Quasi-Anführungszeichen, wie in Abbildung 2 definiert, ersetzt Prolog-Variablen, die durch seine Argumente (`html(Date)`) angegeben werden, wenn sie als Wert für ein Attribut oder einen Inhalt eines Elements angezeigt werden. Es führt die folgenden Schritte aus:

`{ }^9$ http://www.w3.org/TR/sparq11-overview/{ }^10$` Dieses Problem kann auch durch den Quasi-Anzahler gelöst werden, indem Singleton-Variablen entfernt werden, die innerhalb des Ergebnisterms des Quasi-Anzahlers erscheinen.


```
"":- use_module(library(http/thread_httpd)).:-
use_module(library(http/http_dispatch)) :- use_module(library(http/html_write)).

server(Port) :- http_server(http_dispatch, [port(Port) ]).

:- http_handler(/, clock, [1].

clock(_Request) :- get_time(Now),
format_time(atom(Date), 'f+', Now),

    reply_html_page(    title('Meine
Digitaluhr'),          {html(Date)}|
<h1>Meine Digitaluhr</h1>

    <p>Es ist jetzt <span class="time">Date</span>    }).""
```

Abb. 1. Beispielwebserver mit eingebettetem HTML.

1. Analysieren Sie den zitierten HTML-Text mit `load.html/3`. Die Option `max.errors( 0 )` bewirkt, dass der Parser eine Syntaxfehler-Ausnahme auslöst und beim ersten Fehler abbricht. Da diese Verarbeitung beim Lesen der Quelle stattfindet, wird der HTML-Syntaxfehler während der Kompilierung gemeldet und enthält Informationen zu Zeile, Zeilenposition und Zeichenanzahl.² Die XML-DOM-Struktur wird rekursiv durchlaufen, und Attribute, die eine Prolog-Variable als Wert haben, oder Inhalte, die mit einer Prolog-Variable aus Syntax übereinstimmen, werden durch diese Variable ersetzt.

Wenn die Seite generiert wird (Abbildung 1), schließt die Bindung der Variablen `Date` die XML-DOM-Struktur ab. Diese Struktur wird von `reply_html_page/3` serialisiert. Die Serialisierung gewährleistet Typsicherheit und die Generierung einer korrekten HTML-Syntax. `reply_html_page/3`

8.2 Einbetten von JavaScript

Der HTML-Quasi-Quoter des vorherigen Abschnitts ließ sich leicht implementieren, da die SWI-Prolog-Infrastruktur bereits einen Parser und Serialisierer für HTML enthält. Für JavaScript haben wir diese nicht. Wir werden dies wahrscheinlich in Zukunft entwickeln, aber hier wollen wir veranschaulichen, dass es möglich ist, einen sicheren Vorlagengensatz zu erreichen, indem wir nur einen Tokeniser verwenden.

² Die HTML-Infrastruktur verfügt über eine globale Option zur Auswahl zwischen HTML- und XHTML-Serialisierung, was bedeutet, dass HTML in der Prolog-Quelle als XHTML an den Client serialisiert werden kann.

```

"":- module(html_quasi_quotations, [ html/4 ]).
:- use_module(library(sgm)):-
use_module(library(apply)):- use_module(library(lists)):-
use_module(library(quasi_quotations)).

:- quasi_quotation_syntax(html).

html (Content, Vars, Dict, DOM) :- include(qq_var(Vars), Dict, QODict),

    with_quasi_quotation_input(
        Content, In,
        load html (In, DOMO,
            [ max_errors (0)      1])).
    xml_content(QQDict, DOMO, DOM).

qq_var(Vars, _=Var) :- member(V, Vars), V = Var, !.

xml_content(Dict, [Name], [Var]) :-
    atom(Name), memberchk (Name=Var,
Dict), !.xml_content(Dict, Content0, Content)
:-
    maplist(xml_content_element(Dict), Content0, Content).

xml_content_element(Dict,
    element(Tag, Attrs0, Content0), element(Tag,
    Attrs, Content)) :- !, maplist(xml_attribute(Dict), Attrs0,
    Attrs), xml_content(Dict, Content0, Content).

xml_content_element(, Element, Element).

xml_attribute(Dict, Attr=Name, Attr=Var) :- memberchk
(Name=Var, Dict), !.xml_attribute(, Attr, Attr).""

```

Abb. 2. Quellcode für den HTML-Quasi-Quoter

Da JavaScript als Teil der HTML-Seitengenerierung erzeugt wird, erzeugt der JavaScript-Quasi-Quoter eine Ausgabe für das HTML-Backend. Er übersetzt das JavaScript in eine Liste von zwei Arten von Elementen: (1) einfache Atome (die im Kontext eines Skriptelements ausgegeben werden) und Aufrufe einer Grammatik `js_expression//1`, die eine Übersetzung nativer Prolog-Daten in JavaScript-Literale gemäß

Tabelle 1 definiert.

Der Quasi-Quoter zerlegt das zitierte Material mit einem ECMAScript-konformen Tokeniser, der mit einer Prolog-Grammatik implementiert wurde. Der Quoter (Abbildung 4) ersetzt Identifikations-

Prolog	JavaScript
number	number
atom	string (escaped using JavaScript syntax)
true	true
false	false
null	null
constantList	array
array	object
literal	literal

Tabelle 1. Prolog-zu-JavaScript-Konvertierung

fier-Token, die mit einer Prolog-Variable übereinstimmen, mit einem Aufruf von `js expression//1` und übersetzt den Rest in einfache Atome. Abbildung 3 zeigt ein gekürztes Prädikat von der SWI-Prolog-Website, das den JavaScript-Quasi-Quoter verwendet, um ein jQuery-Widget namens `tagit` zu initialisieren. Das eigentliche Prädikat enthält ein größeres Konfigurationsobjekt, nur „mehr vom selben“. In diesem Beispiel wird der erste Block verwendet, um Server-URLs und Eigenschaften für `Obj` zu berechnen, die wir im Rest benötigen. Als Nächstes erstellt der Aufruf `html//1` das für das Tagit-Widget benötigte DOM. Hier hätten wir auch den HTML-Quasi-Quoter verwenden können. Die Wahl ist in diesem Fall eher willkürlich, da das Fragment kurz ist und nur Struktur und Daten definiert, die von einer Variablen übergeben werden. Als Nächstes sehen wir `html requires//1`, was sicherstellt, dass der Seitenkopf erweitert wird, um die erforderlichen JavaScript- und CSS-Ressourcen zu laden, und schließlich gibt es das Skriptfragment mit eingebetteten Prolog-Variablen. Da Prolog-Variablen gültige JavaScript-Bezeichner sind, enthält das Fragment gültige JavaScript-Syntax.

8.3 Einbetten von SPARQL

SPARQL¹² ist die Abfragesprache für die Semantic-Web-RDF-Sprache. In den meisten Abschnitten kann man SPARQL durch SQL ersetzen. ClioPatria ist das Semantic-Web-Framework von SWI-Prolog, das einen SPARQL-„Endpunkt“ enthält. Die SPARQL-Engine kompiliert eine SPARQL-Abfrage in eine Prolog-Abfrage, optimiert und führt diese Abfrage aus und serialisiert die Ergebnisse gemäß der SPARQL-Ergebnisspezifikation. Beim Schreiben von Middleware (Reasoning) in ClioPatria verwendet man typischerweise direkte Abfragen an den eingebetteten RDF-Store für das Reasoning. In einigen Fällen möchte man SPARQL zur Spezifizierung der Abfrage verwenden. Betrachten Sie Fälle, in denen die Abfrage bereits in SPARQL verfügbar ist, die Abfrage auch mit externen Servern verwendet werden muss oder der Autor SPARQL viel besser beherrscht als Prolog. Die Quasi-Quotation-Syntax kann verwendet werden, um den untenstehenden Code aufzuschreiben. Die SPARQL-Abfrage wird zur Kompilierzeit geparkt und die SPARQL-Projektionsvariablen können auf natürliche Weise in die Prolog-Variable der einschließenden Klausel integriert werden. Beachten Sie, dass wir es in diesem Beispiel für besser halten, die substituierten Variablen nicht als Argumente an den sparql-Syntaxterm zu übergeben, da diese Liste explizit in der Variablenprojektionsklausel angegeben wird, die die Abfrage startet.

¹² <http://www.w3.org/TR/sparql11-overview/>

```

prolog:doc_object_page_footer(Obj, _Options) -->
{ http_link_to_id(complete_tag, [], Complete),
  http_link_to_id(show_tag, [],
  OnClick),http_link_to_id(remove_tag, [],
  Remove),object_id(Obj,
  ObjectID),object_tags(Obj,
  Tags),atomic_list_concat(Tags, ' ',
  Data)},html(div(class('user-annotations'),

      input([id(tags), value(Data)]))),
html_requires(tagit),js_script(||javascript(Complete, OnClick,
ObjectID, Remove)||
    $(document).ready(function() {
        $("#tags").tagit({
            autocomplete: { delay: 0.3,
                           minLength:
                           1,source:
                           Complete},
            onTagClicked: function(event, ui) {
                window.location.href = OnClick+"tag="+
                encodeURIComponent(ui.tagLabel);},beforeTa
                gRemoved: function(event, ui) {

                    $.ajax({ dataType: "json",
                           url: Remove,data: { tag:
                           ui.tagLabel,
                           obj: ObjectID}

                    });

                });});});|}
).

```

Abb. 3. Verkürztes Codefragment von der SWI-Prolog-Website, das die Einbettung von JavaScript zur Initialisierung eines Widgets veranschaulicht.

```

...,{|sparql|
SELECT ?Name, ?Place WHERE {
    ...
}|}>

```

```

""javascript(Inhalt, vars, dict, \parts) :-
    include(qq_var(Vars), Dict, QODict),
    phrase_from_quasi_quotation( js(QODict, Teile),
    Inhalt).

K_war(Krieg, _=Krieg): Mitglied(v, Krieg), v=Krieg, !.
Js(Dick, [Vor, Substanz|MEHR]) --> hier(hier0),
js_token(., hier(hier1), js_token(identifizier(name))), {
memberch(name=war, dick), !,
subst=\js_expression(ver), diff_to_atom(here0, here1,
pre)

    }, js(Diktat, Mehr).js(, [Letzte]) -->
Zeichenfolge (Codes), \+ L_, !, {
atom_codes (Letzte, Codes) } .

js_tokens([]) --> [].js_tokens([H|T]) --> js_token(H), js_tokens(T).

! diff_to_atom(+Start, +Ende, -Atom)%

i Wahr, wenn Atom ein Atom ist, das seine Zeichen zwischen Start
und Ende darstellt, wobei End am Ende der Liste Start stehen muss.

diff_to_atom(Start, Ende, Atom) :- diff_list(Start, Ende, Liste),

    atom_codes (Atom, Liste) .

diff_list(Start, Ende, Liste) :- Start == Ende, !, Liste = [].diff_list([H|Start],
Ende, [h|Liste]) :- diff_list(Start, Ende, Liste).

hier (hier, hier, hier)."

```

Abb. 4. Teilquelle für das Javascript-Syntax-Zitat

Ein ähnlicher Ansatz ist mit SQL machbar und bietet eine bequemere Möglichkeit zur Interaktion mit der Datenbank als die Verwendung vorbereiteter Anweisungen und sowohl eine sicherere als auch eine bequemere Methode als direkte textbasierte SQL-Abfragen.

9 Umsetzung

Die Implementierung der quasi-Angebotsabwicklung in ein bestehendes Prolog-System ist unkompliziert und sollte nicht länger als ein paar Tage in Anspruch nehmen. Der Tokenisierer wird erweitert, um `$_{\mid}` zu erkennen. Wenn der Parser auf dieses Token trifft, erstellt er eine Liste von Begriffen, von denen jeder ein Quasi-Zitat darstellt. Wenn die Liste der Quasi-Zitate nicht leer ist, wenn `read/1` das Fullstop-Token liest, materialisiert es das Variablenwörterbuch und ruft eine Routine auf, die die Quasi-Zitat-Parser aufruft. Beachten Sie, dass Quasi-Zitate nach dem Lesen von `$_{\mathbf{}}` nach Abschluss des Lesens des Begriffs geparkt werden. Dies ermöglicht es, dass Quasi-Zitate auf Variablen verweisen, die nach dem Quasi-Zitat erscheinen (siehe Abschnitt 8.3). Dies impliziert, dass `read $_{\mathbf{1}}` den zitierten Text puffern muss. Die Funktionalität wird dem Benutzer über die Bibliotheks `quas1.quotations.pl` zur Verfügung gestellt, die in SWI-Prolog 6.3.17 enthalten ist.

Der Prolog-Tokenisierer wird um zwei Token erweitert. `$_{\mid}` ist ein neuer Token, der eine Quasi-Notierung beginnt. Die Sequenz `$_{\ldots}` wird als einzelnes Token verarbeitet.

10 Zukünftige Arbeiten

Quasi-Zitate sind in SWI-Prolog noch jung. Wir sind der Meinung, dass die in diesem Artikel beschriebene Kernfunktionalität weitgehend zukunftssicher ist.

Eine der zukünftigen Aufgaben ist die Etablierung von Bibliotheken, die die Handhabung von Syntax und die sichere Ersetzung erleichtern. Eine andere besteht darin, Entwurfsmuster für den Einsatz dieser Technologie zu etablieren. Was kann man hier von den Haskell-, Python- und Ruby-Communities lernen? Quasi-Anführungszeichen werden in der Regel zu einem Prolog-Begriff "ausgewertet", der Variablen mit der Klausel teilt, in die er eingebettet ist. Wir haben keine funktionale Syntax, die wir ausnutzen können, um die "Auswertung" dieses Begriffs mit instanziierten Variablen zu erzwingen. Auf der anderen Seite können wir auch den abstrakten Syntaxbaum ohne Boden verwenden und ihn z.B. für Matching-Aufgaben verwenden.

11 Schlussfolgerungen

In diesem Artikel haben wir motiviert, warum es notwendig ist, längere Code-Schnipsel, die in einer anderen Sprache geschrieben wurden, in den Prolog-Quellcode einbetten zu können. Dies erfordert eine Syntax, die das Einbetten ermöglicht, praktisch ohne dass Prolog in den eingebetteten Text eingebettet werden muss. Als Nächstes muss dieser Text mit Prolog-Daten verknüpft werden. Die Integration von Prolog-Daten in das Snippet muss entsprechend der Syntax und dem Datenmodell der externen Sprache erfolgen. Dies bedeutet, dass wir eine programmierbare Komponente benötigen, die sich auf das angebotene Material bezieht. Die Kombination dieser Anforderungen in einer syntaktischen Erweiterung, die von `read/1` ausgeführt wird, vereinfacht die Unterstützung durch Tools wie Editoren, bietet einen natürlichen Zugriff auf Prolog-Variablen und ermöglicht eine nahtlose Integration von Fehlermeldungen. Wir haben Quasi-Angebote mit zwei implementierten Quotierern demonstriert. Diese Anführungszeichen ermöglichen das einfache Kopieren und Einfügen von Material in ihrer nativen Syntax in und aus der Prolog-Quelle. Die Verwendung dieser Zitate reduziert wahrscheinlich die Lernkurve für das Einbetten von Snippets in Prolog, während der Quasi-Zitator garantieren kann, dass das integrierte Material (syntaktisch) korrekt ist und dass das Prolog-Material vollständig integriert ist.

Danksagung Jan Pobrislo lieferte die Erkenntnisse, die für das Schreiben von Abschnitt 5 verwendet wurden. Michiel Hildebrand und Jacco van Ossenbruggen (VU University Amsterdam) haben viele der reinen Prolog-basierten Alternativen ausprobiert, die für die Formulierung von Abschnitt 3 und Abschnitt 4 benötigt wurden. Sie gestalten auch den aktuellen JavaScript-Quasi-Zitatgeber mit und kommentieren Entwürfe dieses Textes. Ulrich Neumerkel hat die Kombination von | mit Klammern und wies auf die in Abschnitt 9 beschriebenen Tokenisierungsprobleme hin. Diese Veröffentlichung wurde unterstützt durch das niederländische nationale Programm COMMIT/

Referenzen

1. Jake Donham und Nicolas Pouillard. Camlp4 und Template Haskell. In ACM SIGPLAN Commercial Users of Functional Programming, CUFPP '10, Seiten 6:1–6:1, New York, NY, USA, 2010. ACM
2. Christoph Draxler. Zugriff auf relationale und höhere Datenbanken durch Datenbankmengen-Prädikate in logikorientierten Programmiersprachen. Dissertation, Universität Zürich, 1991.
3. Daniel Cabeza Gras und Manuel V. Hermenegildo. Verteilte WWW-Programmierung mit (cizo-)Prolog und der piLLOW-Bibliothek. TPLP, 1(3):251–282, 2001.
4. Geoffrey Mainland. Warum es schön ist, zitiert zu werden: Quasiquoting für Haskell. In Gabriele Keller, Hrsg., Haskell, Seiten 73–82. ACM, 2007.
5. Jan Wielemaker und Nicos Angelopoulos. Syntaktische Integration externer Sprachen in Prolog. In Proceedings of WLPE 2012, 2012.
6. Jan Wielemaker, Zhisheng Huang und Lourens van der Meij. Swi-Prolog und das Web. TPLP, 8(3):363–392, 2008.
7. Jan Wielemaker und Ulrich Neumerkel. Präzises Garbage Collection in Prolog. In Proceedings of CICLOPS 2008, 2008.